GPU프로그래밍

프로젝트 발표

202210946 유시은
202210229 양효인

목차

table of contents

1

복도 및 강의실 공간 구현

복도 및 강의실 공간 구현

벽면 texture 로드하기

```
// build and compile shaders
// -------------------------
Shader shader("shader/wall.vs", "shader/wall.fs");
```

```
// load textures
// -------------
unsigned int floorTexture = loadTexture("resources/textures/floor.png");
unsigned int wallTexture = loadTexture("resources/textures/wall.png");
unsigned int ceilingTexture = loadTexture("resources/textures/ceiling.png");

// shader configuration
// --------------------
shader.use();
shader.setInt("texture1", 0);
```

wall.vs

```
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec2 aTexCoords;

out vec2 TexCoords;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main()
{
    TexCoords = aTexCoords;
    gl_Position = projection * view * model * vec4(aPos, 1.0);
}
```

wall.fs

```
#version 330 core
out vec4 FragColor;

in vec2 TexCoords;

uniform sampler2D texture1;

void main()
{
    vec4 texColor = texture(texture1, TexCoords);
    FragColor = texColor;
}
```

Part 1

복도 및 강의실 공간 구현

각 벽면에 대한 x, y, z 좌표 설정 및 texture 반복 횟수 설정

```
// Front wall vertices
float wallFrontVertices[] = {
    // positions          // texture Coords
    // Front face
    -7.5f, -1.0f,  7.5f,  0.0f, 0.0f,
     7.5f, -1.0f,  7.5f,  10.0f, 0.0f,
     7.5f,  2.0f,  7.5f,  10.0f, 10.0f,
    -7.5f, -1.0f,  7.5f,  0.0f, 0.0f,
     7.5f,  2.0f,  7.5f,  10.0f, 10.0f,
    -7.5f,  2.0f,  7.5f,  0.0f, 10.0f
};
```

```
// wallFront VAO
unsigned int wallVAO, wallVBO;
glGenVertexArrays(1, &wallVAO);
glGenBuffers(1, &wallVBO);
glBindVertexArray(wallVAO);
glBindBuffer(GL_ARRAY_BUFFER, wallVBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(wallFrontVertices), wallFrontVertices, GL_STATIC_DRAW);
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 5 * sizeof(float), (void*)0);
glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 5 * sizeof(float), (void*)(3 * sizeof(float)));
```

```
shader.use();
shader.setMat4("view", view);
shader.setMat4("projection", projection);

// draw floor as normal, but don't write t
glStencilMask(0x00);
```

Part 1

복도 및 강의실 공간 구현

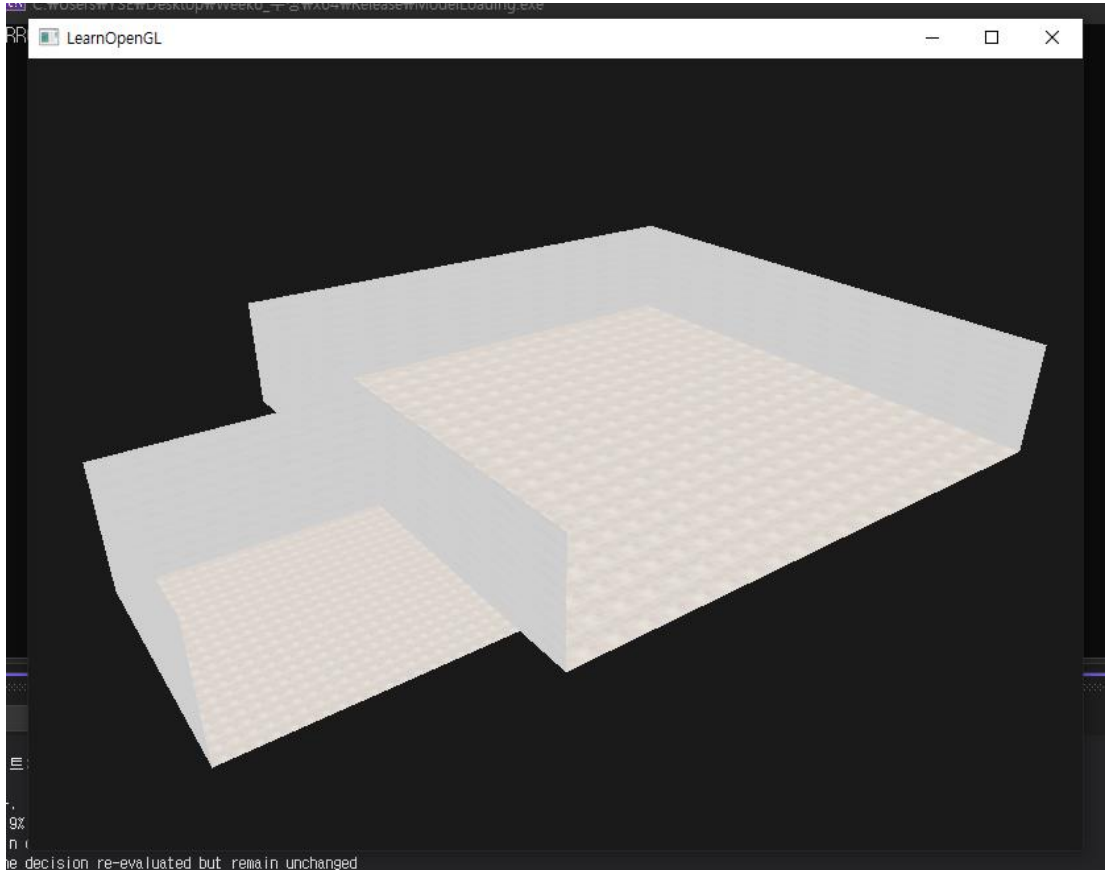
render loop

```
shader.use();
shader.setMat4("view", view);
shader.setMat4("projection", projection);

// draw floor as normal, but don't write t
glStencilMask(0x00);
```

```
glBindVertexArray(wallRightVAO);
glBindTexture(GL_TEXTURE_2D, wallTexture);
shader.setMat4("model", glm::mat4(1.0f));
glDrawArrays(GL_TRIANGLES, 0, 6);
glBindVertexArray(0);
```

복도 직육면체에서 6개의 면, 교실 직육면체에서 6개의 면,
총 12개의 벽면을 바인딩하기
(구현 결과에서 편의상 일부만 확인하게 함)

구현 결과



2

모델 선정 및 배치

Part 2

모델선정및배치

Assimp 환경 빌드하고 3D모델 파일 불러오기

```
Shader ourShader("shader/1.model_loading.vs", "shader/1.model_loading.fs");

Model computer("resources/computer/scene.gltf");
Model door("resources/door/scene.gltf");
Model whiteboard("resources/whiteboard/scene.gltf");
Model clock("resources/clock/scene.gltf");
Model table("resources/table/scene.gltf");
Model chair("resources/chair/scene.gltf");
Model smallTree("resources/pot/scene.gltf");
Model glass_window ("resources/glass_window/scene.gltf");
Model diner_door("resources/diner_door/scene.gltf");
Model tree("resources/bamboo_with_plant_pot/scene.gltf");
Model schoolLock("resources/school_locker/scene.gltf");
Model blind("resources/blind/scene.gltf");
Model windows("resources/window/scene.gltf");
Model screen("resources/screen/scene.gltf");
Model air_conditioner("resources/air_conditioner/scene.gltf");
Model front_table("resources/front_table/scene.gltf");
```

1.model_loading.fs

```
#version 330 core
out vec4 FragColor;

in vec2 TexCoords;

uniform sampler2D texture_diffuse1;

void main()
{
    FragColor = texture(texture_diffuse1, TexCoords);
}
```

1.model_loading.vs

```
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;
layout (location = 2) in vec2 aTexCoords;

out vec2 TexCoords;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main()
{
    TexCoords = aTexCoords;
    gl_Position = projection * view * model * vec4(aPos, 1.0);
}
```

Part 2

모델선정및배치

render loop

```
ourShader.use();

// view/projection transformations
glm::mat4 projection = glm::perspective(glm::radians(camera.Zoom), (float)SCR_WIDTH / (float)SCR_HEIGHT, 0.1f, 100.0f);
glm::mat4 view = camera.GetViewMatrix();
ourShader.setMat4("projection", projection);
ourShader.setMat4("view", view);
```

```
glm::mat4 smallTreePot = glm::mat4(1.0f);
smallTreePot = glm::translate(smallTreePot, glm::vec3(-2.0f, -1.0f, 2.0f));
smallTreePot = glm::scale(smallTreePot, glm::vec3(0.7f, 0.7f, 0.7f));
smallTreePot = glm::rotate(smallTreePot, glm::radians(90.0f), glm::vec3(0.0, 1.0, 0.0));
ourShader.setMat4("model", smallTreePot);
smallTree.Draw(ourShader);
```

translate/scale/rotate 함수를 적절히 활용하여 모델 배치

Part 2

모델 선정 및 배치

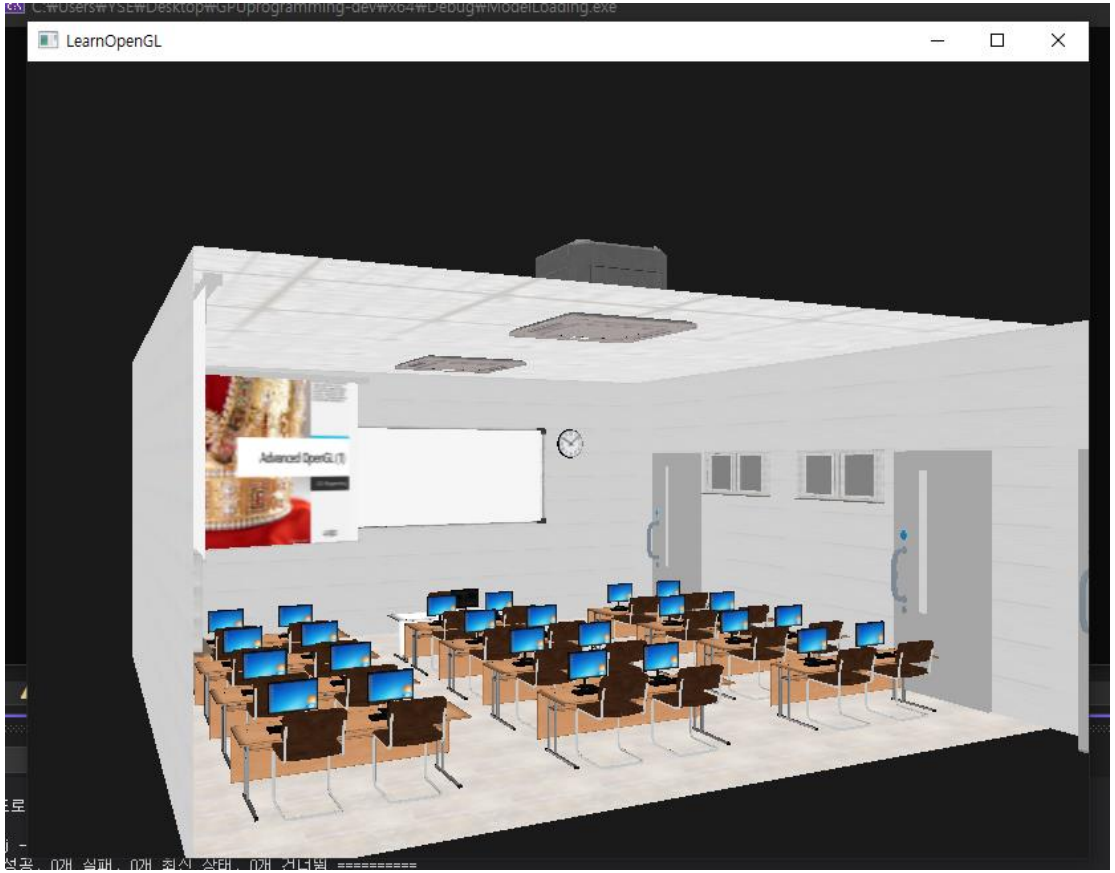
```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        glm::mat4 model_table = glm::mat4(1.0f);
        model_table = glm::translate(model_table, glm::vec3(-12.5f + i * 2, -1.0f, 2.0f + j * 1));  // translate it dow
        model_table = glm::scale(model_table, glm::vec3(1.1f, 0.8f, 0.8f));  // it's a bit too big for our scene, so sc
        model_table = glm::rotate(model_table, glm::radians(90.0f), glm::vec3(-1.0, 0.0, 0.0));
        model_table = glm::rotate(model_table, glm::radians(90.0f), glm::vec3(0.0, 0.0, -1.0));
        ourShader.setMat4("model", model_table);
        table.Draw(ourShader);

        glm::mat4 model_left_computer = glm::mat4(1.0f);
        model_left_computer = glm::translate(model_left_computer, glm::vec3(-12.8f + i * 2, -0.45f, 1.97f + j * 1)); //
        model_left_computer = glm::scale(model_left_computer, glm::vec3(0.2f, 0.2f, 0.2f)); // it's a bit too big for c
        ourShader.setMat4("model", model_left_computer);
        computer.Draw(ourShader);

        glm::mat4 model_right_computer = glm::mat4(1.0f);
        model_right_computer = glm::translate(model_right_computer, glm::vec3(-12.2f + i * 2, -0.45f, 1.97f + j * 1));
        model_right_computer = glm::scale(model_right_computer, glm::vec3(0.2f, 0.2f, 0.2f));   // it's a bit too big f
        ourShader.setMat4("model", model_right_computer);
        computer.Draw(ourShader);

        glm::mat4 model_left_chair = glm::mat4(1.0f);
        model_left_chair = glm::translate(model_left_chair, glm::vec3(-12.8f + i * 2, -0.7f, 2.2f + j * 1)); // transla
        model_left_chair = glm::scale(model_left_chair, glm::vec3(0.3f, 0.3f, 0.3f));   // it's a bit too big for our s
        model_left_chair = glm::rotate(model_left_chair, glm::radians(180.0f), glm::vec3(0.0, 1.0, 0.0));
        ourShader.setMat4("model", model_left_chair);
        chair.Draw(ourShader);

        glm::mat4 model_right_chair = glm::mat4(1.0f);
        model_right_chair = glm::translate(model_right_chair, glm::vec3(-12.2f + i * 2, -0.7f, 2.2f + j * 1)); // trans
        model_right_chair = glm::scale(model_right_chair, glm::vec3(0.3f, 0.3f, 0.3f)); // it's a bit too big for our s
        model_right_chair = glm::rotate(model_right_chair, glm::radians(180.0f), glm::vec3(0.0, 1.0, 0.0));
        ourShader.setMat4("model", model_right_chair);
        chair.Draw(ourShader);
    }
}
```

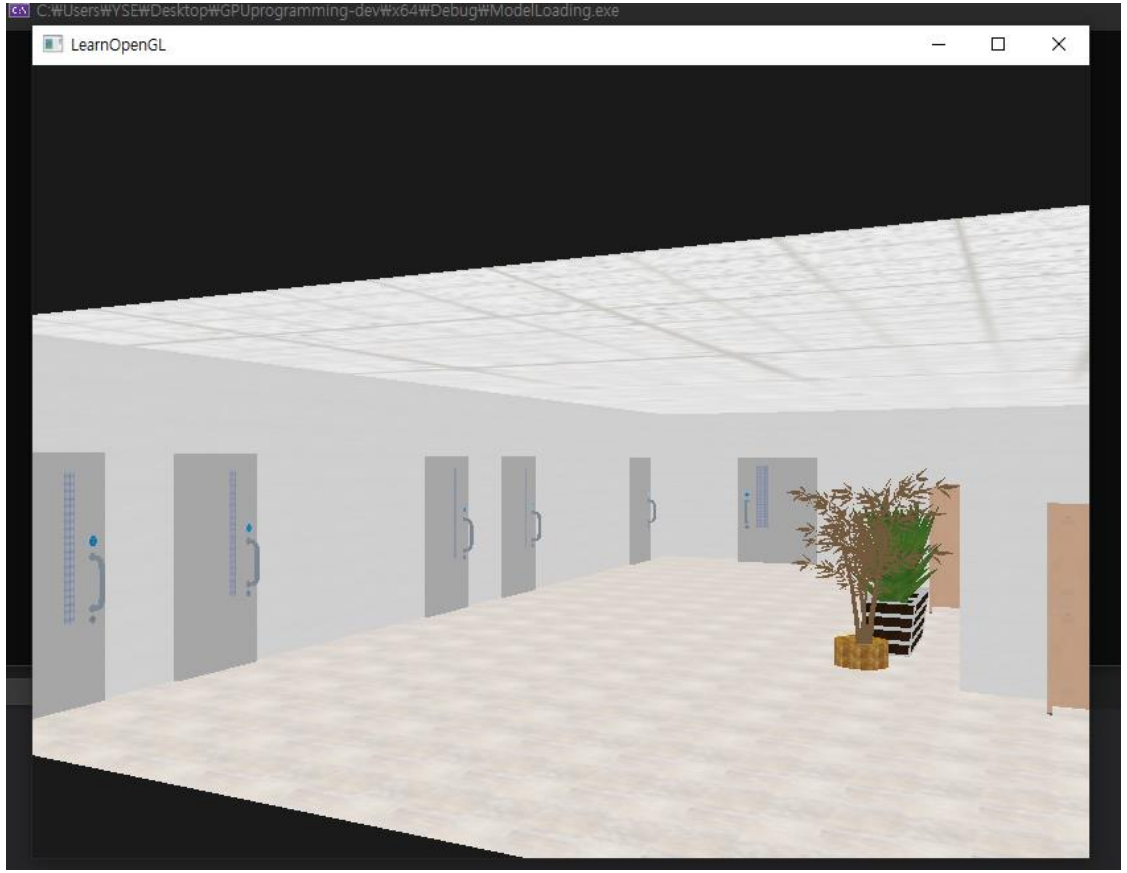
for문을 활용하여 반복되는 여러 모델을 효율적으로 배치할 수 있게 함

Part 2

모델 선정 및 배치

구현 결과





3

카메라 시점 제한

Part 3

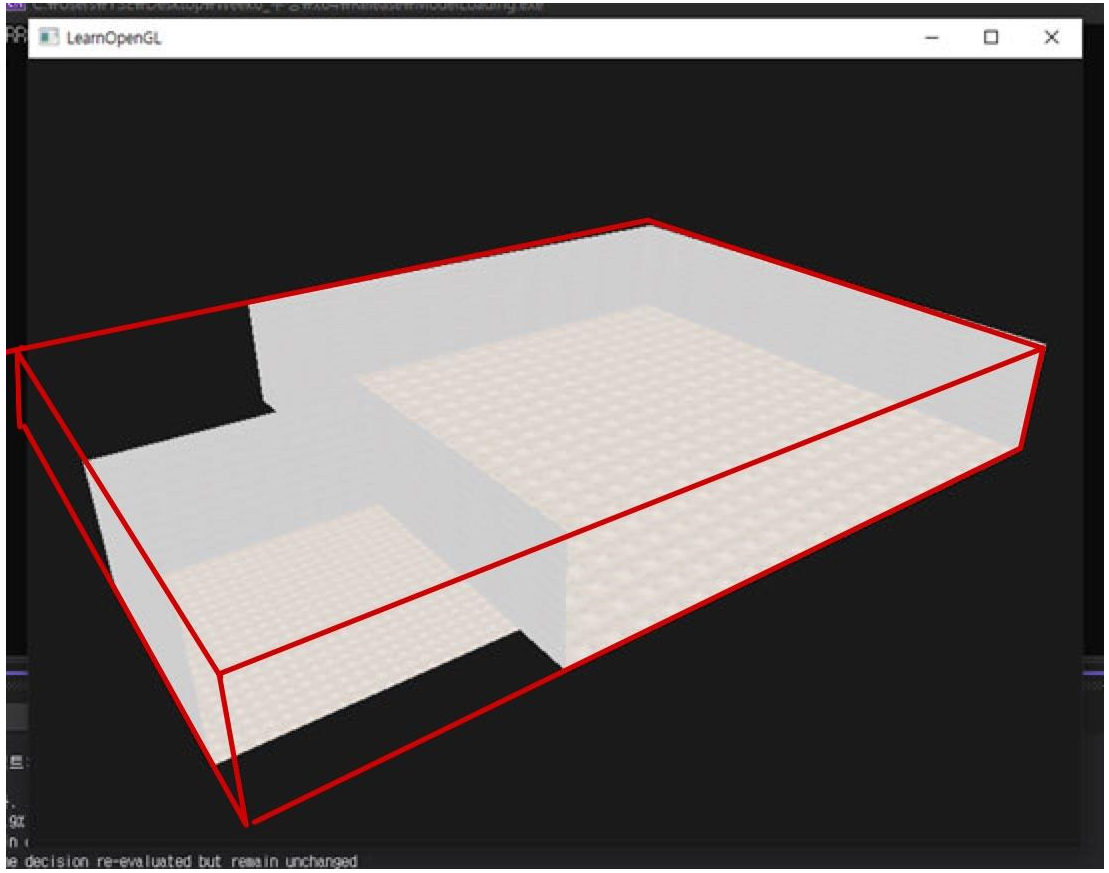
카메라시점제한

camera.h 수정하여 x, z축 제한 범위 설정

```
// camera
Camera camera(glm::vec3(5.0f, 0.0f, -5.0f));
float lastX = SCR_WIDTH / 2.0f;
float lastY = SCR_HEIGHT / 2.0f;
bool firstMouse = true;
```

```
void ProcessKeyboard(Camera_Movement direction, float deltaTime)
{
    // Define the movement limits for the vertices
    float minX = -13.5f;
    float maxX = 7.5f;
    float minZ1 = 0.0f;
    float maxZ1 = 6.0f;
    float minZ2 = -7.5f;
    float maxZ2 = 7.5f;
```

camera.h



4

Kernel effect 구현

Part 4

Kernel effect 구현

shader.h에 배열 uniform 값을 설정할 수 있도록 함수 추가

shader.h

```
void setFloatArray(const std::string& name, const float* values, int count)
{
    glUniform1fv(glGetUniformLocation(ID, name.c_str()), count, values);
}
```

5.1.framebuffers_screen.fs

```
#version 330 core
out vec4 FragColor;

in vec2 TexCoords;

uniform sampler2D screenTexture;
const float offset = 1.0 / 300.0;
uniform float kernel[9];

void main()
{
    vec2 offsets[9] = vec2[](
        vec2(-offset,  offset), // top-left
        vec2( 0.0f,    offset), // top-center
        vec2( offset,  offset), // top-right
        vec2(-offset,  0.0f),   // center-left
        vec2( 0.0f,    0.0f),   // center-center
        vec2( offset,  0.0f),   // center-right
        vec2(-offset, -offset), // bottom-left
        vec2( 0.0f,   -offset), // bottom-center
        vec2( offset, -offset)  // bottom-right
    );
```

5.1.framebuffers_screen.fs

```
    vec3 sampleTex[9];
    for(int i = 0; i < 9; i++)
    {
        sampleTex[i] = vec3(texture(screenTexture, TexCoords.st + offsets[i]));
    }
    vec3 col = vec3(0.0);
    for(int i = 0; i < 9; i++)
        col += sampleTex[i] * kernel[i];

    FragColor = vec4(col, 1.0);
}
```

5.1.framebuffers_screen.vs

```
#version 330 core
layout (location = 0) in vec2 aPos;
layout (location = 1) in vec2 aTexCoords;

out vec2 TexCoords;

void main()
{
    TexCoords = aTexCoords;
    gl_Position = vec4(aPos.x, aPos.y, 0.0, 1.0);
}
```

Part 4

Kernel effect 구현

kernel 배열 전역변수 선언

```
float kernel[9] = {
    0.0f, 0.0f, 0.0f,
    0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 0.0f };
```

```
// kernel effects

Shader screenShader("shader/5.1.framebuffers_screen.vs", "shader/5.1.framebuffers_screen.fs");
screenShader.use();
screenShader.setInt("screenTexture", 0);

float quadVertices[] = { // vertex attributes for a quad that fills the entire screen in Normalized Device Coordinates.
    // positions   // texCoords
    -1.0f,  1.0f,  0.0f, 1.0f,
    -1.0f, -1.0f,  0.0f, 0.0f,
     1.0f, -1.0f,  1.0f, 0.0f,

    -1.0f,  1.0f,  0.0f, 1.0f,
     1.0f, -1.0f,  1.0f, 0.0f,
     1.0f,  1.0f,  1.0f, 1.0f
};
```

Part 4

Kernel effect 구현

```
// screen quad VAO
unsigned int quadVAO, quadVBO;
glGenVertexArrays(1, &quadVAO);
glGenBuffers(1, &quadVBO);
glBindVertexArray(quadVAO);
glBindBuffer(GL_ARRAY_BUFFER, quadVBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(quadVertices), &quadVertices, GL_STATIC_DRAW);
glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float), (void*)0);
glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 4 * sizeof(float), (void*)(2 * sizeof(float)));

// framebuffer configuration
// -------------------------
unsigned int framebuffer;
glGenFramebuffers(1, &framebuffer);
glBindFramebuffer(GL_FRAMEBUFFER, framebuffer);
// create a color attachment texture
unsigned int textureColorbuffer;
glGenTextures(1, &textureColorbuffer);
glBindTexture(GL_TEXTURE_2D, textureColorbuffer);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, SCR_WIDTH, SCR_HEIGHT, 0, GL_RGB, GL_UNSIGNED_BYTE, NULL);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, textureColorbuffer, 0);
// create a renderbuffer object for depth and stencil attachment (we won't be sampling these)
unsigned int rbo;
glGenRenderbuffers(1, &rbo);
glBindRenderbuffer(GL_RENDERBUFFER, rbo);
glRenderbufferStorage(GL_RENDERBUFFER, GL_DEPTH24_STENCIL8, SCR_WIDTH, SCR_HEIGHT); // use a single
glFramebufferRenderbuffer(GL_FRAMEBUFFER, GL_DEPTH_STENCIL_ATTACHMENT, GL_RENDERBUFFER, rbo); // nov
// now that we actually created the framebuffer and added all attachments we want to check if it is
if (glCheckFramebufferStatus(GL_FRAMEBUFFER) != GL_FRAMEBUFFER_COMPLETE)
    std::cout << "ERROR::FRAMEBUFFER:: Framebuffer is not complete!" << std::endl;
glBindFramebuffer(GL_FRAMEBUFFER, 0);
```

Part 4

Kernel effect 구현

render loop

```
screenShader.use();
screenShader.setFloatArray("kernel", kernel, 9);
glBindVertexArray(quadVAO);
glBindTexture(GL_TEXTURE_2D, textureColorbuffer);
glDrawArrays(GL_TRIANGLES, 0, 6);
```

```
// bind to framebuffer and draw scene as we normally would to color texture
glBindFramebuffer(GL_FRAMEBUFFER, framebuffer);
glEnable(GL_DEPTH_TEST); // enable depth testing (is disabled for rendering screen-space quad)
```

key입력해 kernel 행렬 입력받기

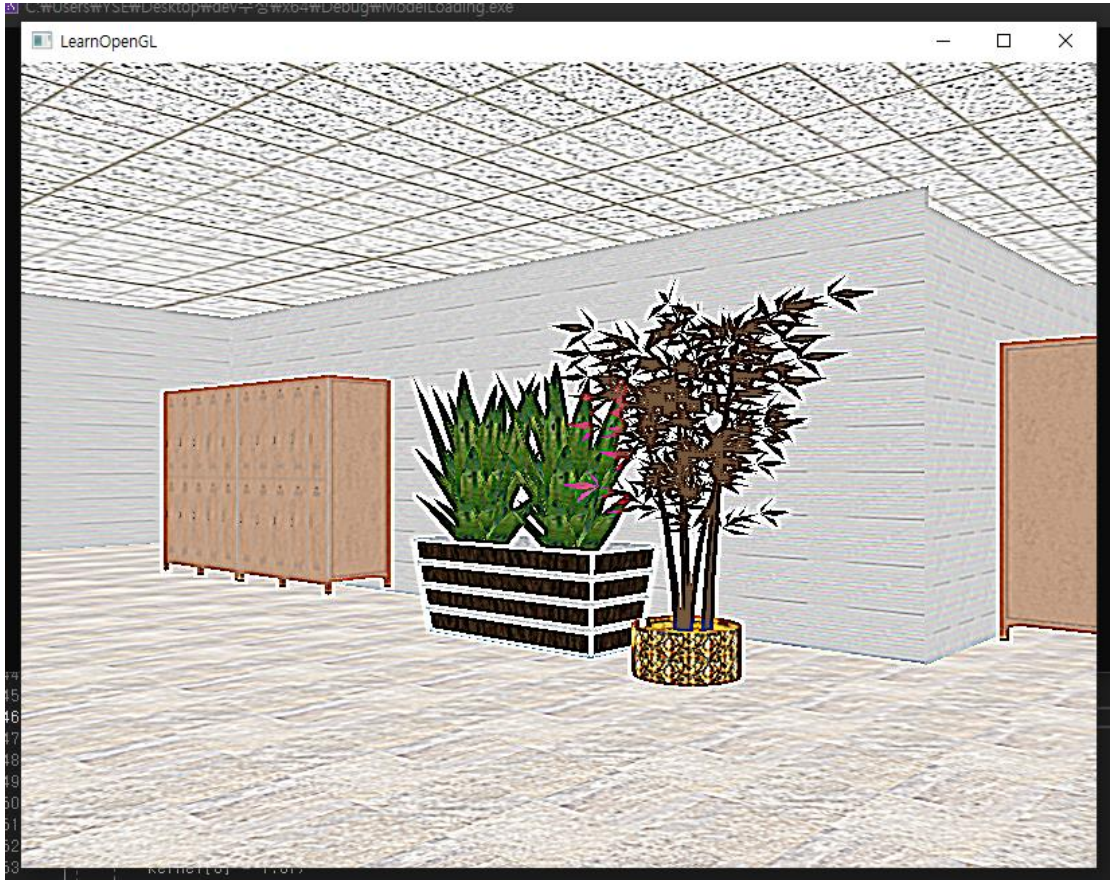
```
void processInput(GLFWwindow* window)
```

```
if (glfwGetKey(window, GLFW_KEY_F1) == GLFW_PRESS)
{
    kernel[0] = -1.0f; kernel[1] = -1.0f; kernel[2] = -1.0f;
    kernel[3] = -1.0f; kernel[4] =  9.0f; kernel[5] = -1.0f;
    kernel[6] = -1.0f; kernel[7] = -1.0f; kernel[8] = -1.0f;
}
```

Part 4

Kernel effect 구현

구현 결과



5

애니메이션 효과 구현

Part 5

애니메이션 효과구현

translate 함수에서 x, z 값을 cos값 변화를 통해
주기적으로 바꿈으로써 애니메이션 효과를 구현함

render loop

```
glm::mat4 girl_walk = glm::mat4(1.0f);
float radius = 5.0f;
float speed = 0.5f;
float angle = glfwGetTime() * speed;
//float x = radius * cos(angle);
float z = radius * cos(angle);
glm::mat4 model = glm::mat4(1.0f);
girl_walk = glm::translate(model, glm::vec3(4.0f, -1.0f, z)); // x와 z 좌표에 따라 이동
girl_walk = glm::scale(girl_walk, glm::vec3(0.0008f, 0.0008f, 0.0008f));
girl_walk = glm::rotate(girl_walk, glm::radians(-90.0f), glm::vec3(1.0, 0.0, 0.0));

ourShader.setMat4("model", girl_walk);
person_walk.Draw(ourShader);


glm::mat4 teacher_walk = glm::mat4(1.0f);
radius = 1.0f;
float x = radius * cos(angle);
//float z = radius * sin(angle);
model = glm::mat4(1.0f);
teacher_walk = glm::translate(model, glm::vec3(x-9.5, -1.0f, 0.9)); // x와 z 좌표에 따라 이동
//teacher_walk = glm::scale(teacher_walk, glm::vec3(5.0f, 5.0f, 5.0f));
teacher_walk = glm::rotate(teacher_walk, glm::radians(-90.0f), glm::vec3(1.0, 0.0, 0.0));
ourShader.setMat4("model", teacher_walk);
professor.Draw(ourShader);
```

구현 결과

Q&A

발표를 들어주셔서 감사합니다